DC Motor Speed Control Using a PI Controller

1.0 Learning Outcomes……1
2.0 Engineering Requirements……1
3.0 Hardware Set Up……2
4.0 Simulink Set Up and Results……..…..…5
5.0 Verification and Validation..……..9
6.0 Concluding Remarks……..…..11
7.0 Equipment Pack Away……....11
Recommended Citation……..……12
Appendix A……..…..13

1.0 Learning Outcomes

After completing this exercise, you should be able to demonstrate the Learning Outcomes listed in Table 1. The exercise follows a structured engineering workflow: Requirements → Implementation → Verification and Validation. At the end of the exercise, revisit the Learning Outcomes in Table 1 and confirm that each has been achieved.

Before starting this exercise, it is important that you have completed the following exercise: DC Motor Speed Measurement Using an Encoder.

Table 1: *Learning Outcomes from the DC Motor Speed Control Using a PI Controller Exercise*

ID	Learning Outcome	Achieved
1.	Interpret the engineering requirements for a closed-loop DC motor speed control system and assemble the required hardware.	☐
2.	Implement a discrete-time PI controller in Simulink, incorporating output saturation, anti-windup and low-pass filtering of the measured motor speed.	☐
3.	Verify and validate the initial closed-loop motor speed response against the defined engineering requirements, identifying where further controller development may be required.	☐

2.0 Engineering Requirements

The engineering requirements for the DC motor speed control system are detailed in Table 2. Once the control system has been implemented and the experimental results obtained, these requirements will be used to verify and validate the performance of the implemented controller.

Table 2: *Engineering Requirements for the DC Motor Speed Control System*

ID	Engineering Requirement
R1	The DC motor shall track the specified reference speed of 30RPM.
R2	The closed-loop response shall achieve approximately zero steady-state error.
R3	The rise time shall be less than 2s.
R4	The controller output shall remain within the permitted actuator range of 0–170, i.e., the permitted actuator range.
R5	The measured speed signal shall be sufficiently filtered to enable stable closed-loop control.

3.0 Hardware Set Up

This exercise builds on the hardware and algorithm setup described in the exercise, DC Motor Speed Measurement using an Encoder, which details the operation of a DC motor with an encoder.

The setup will be repeated here for completeness.

When setting-up the hardware and using the ACE-Lab (Base + Sense, Actuate), it is advised you place them side-by-side and use the relevant locations for the actuate components, see Figure 1.

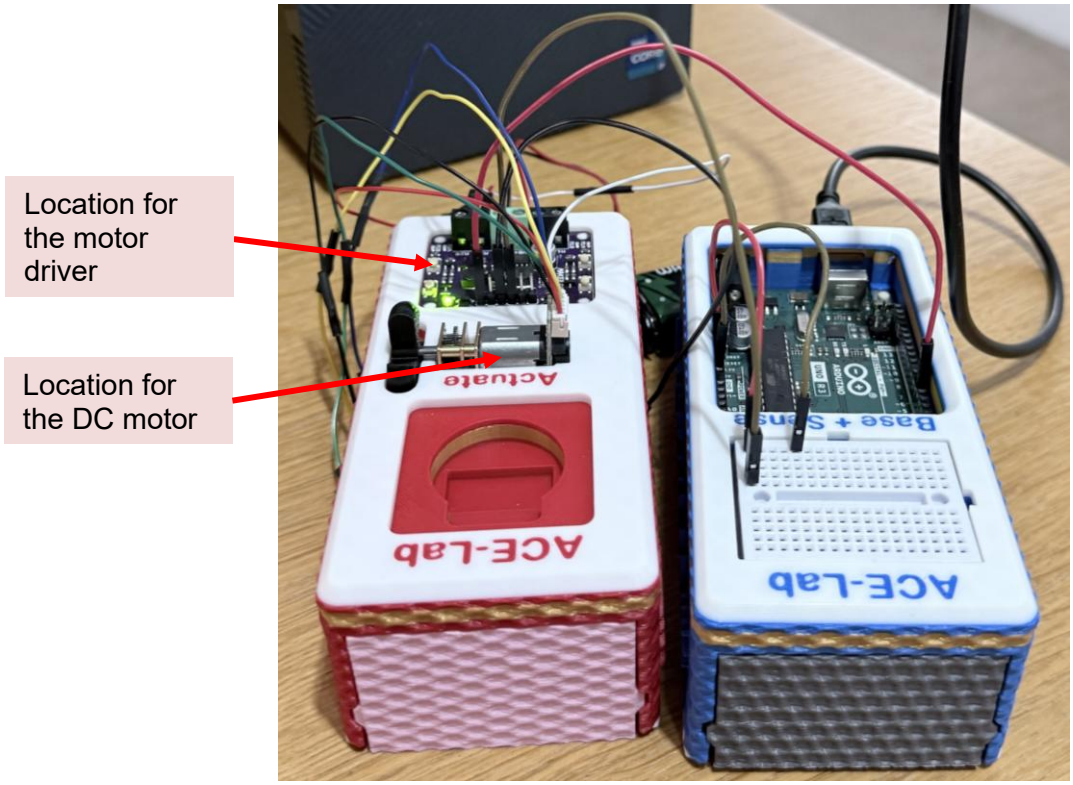


Figure 1: *Use of ACE-Lab (Base + Sense, Actuate) Together, Side-by-Side*

The Arduino provides the control signal to the Cytron Maker Drive motor driver, which regulates the power supplied to the motor. A $9V$ external supply powers the motor driver, while the DC motor itself is rated at $6V$. The motor encoder generates pulses as the motor rotates, which are measured by the Arduino and used to estimate the motor speed. As in the previous exercise, the Arduino and motor driver must share a common ground to ensure that the control signals are correctly referenced and recognised.

Required Hardware

For this exercise, the following hardware is required:

1. ATmega328P microcontroller
2. Breadboard
3. Cytron Maker Drive motor driver board
    a. Supply voltage: 2.5 – 9.5V DC
    b. Maximum continuous current: 1.0A
4. DC motor with encoder and 1:298 gearbox
    a. Rated voltage: 6V DC
    b. Rated current: approximately 0.3A
5. 9V battery clip
6. 9V battery
7. Breadboard jumper wires
    a. 2 × male-to-male wires (10cm)
    b. 3 × male-to-female wires (20cm)
8. USB Type-B cable

Step-by-step set-up instructions

To set up the hardware, use the circuit diagram provided in Figure 2 as a reference. A combination of male-to-male (m-m) jumper wires, supplied with the ACE-Lab (Base + Sense), and male-to-female (m-f) jumper wires, supplied with the ACE-Lab (Actuate), will be required.

The hardware should be connected as follows:

1. Connect the two DC motor wires to the Maker Drive motor output terminals for the selected motor channel.
2. Connect the positive terminal of the 9 V battery to the Maker Drive motor power input, labelled VB+.
3. Connect the negative terminal of the 9 V battery to the Maker Drive ground terminal, labelled VB−/GND.
4. Connect one of the Arduino GND pins to the Maker Drive GND connection. This creates a common ground between the Arduino and motor driver, ensuring that the control signal has the same voltage reference.
5. Connect the encoder VCC (+5 V) connection to the Arduino 5 V supply.
6. Connect the encoder GND connection to an Arduino GND connection. This should form part of the same common ground used by the Arduino and motor driver.
7. Connect encoder Channel A to Arduino digital pin 2.
8. Connect encoder Channel B to Arduino digital pin 3. Channels A and B provide the encoder signals required to determine the

direction and rotational speed of the motor. These pin assignments correspond with the Simulink encoder configuration used later in the exercise.

9. Connect Arduino Pin 5 to the appropriate Maker Drive control input for the selected motor channel, as shown in Figure 2. Pin 5 provides the PWM control signal used to regulate the voltage applied to the motor.
10. Ensure that the second Maker Drive control input for the selected motor channel is connected or configured as shown in Figure 2, so that the required direction of motor rotation is obtained.
11. Where convenient, use the breadboard power rails to organise and distribute the 5 V and GND connections required by the encoder.
12. Finally, connect the Arduino to the computer using the USB Type-B cable.

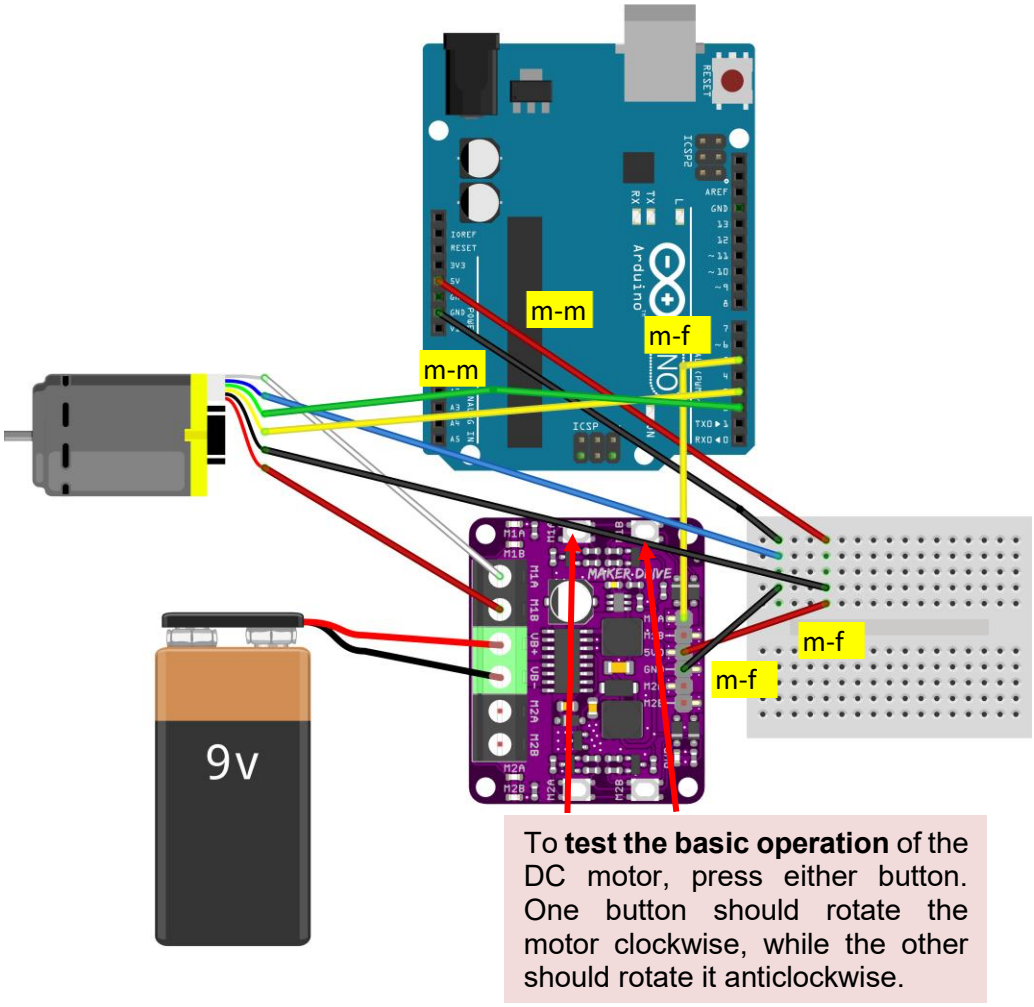


Figure 2: *Hardware Set Up for DC Motor Speed Measurement using Encoder*

4.0 Simulink Set Up and Results

Building on the previous *DC Motor Speed Measurement using an Encoder* exercise, the encoder signal is converted into motor speed using the existing Unit Delay and Gain blocks. It is therefore recommended that the previous exercise is completed first or referred back to where support is needed in locating and configuring Simulink blocks. At this stage of the ACE-Lab exercises, however, you are expected to be increasingly confident in constructing and configuring Simulink diagrams independently.

Recall: Double-click the Encoder block and set the sample time to 0.01s.

A Discrete Filter block is then added to reduce high-frequency noise in the measured speed signal. Double-click the Discrete Filter block and enter the numerator and denominator coefficients shown in Figure 3. These coefficients determine both the degree of smoothing and the speed of the filter response. A filter that responds too quickly may allow excessive measurement noise to pass through, whereas a filter that responds too slowly may introduce unnecessary delay and cause the measured speed to lag the actual motor response. As shown in Figure 3, the blocks have been flipped to improve the layout of the Simulink diagram. To flip a block, select it and press Ctrl + I.

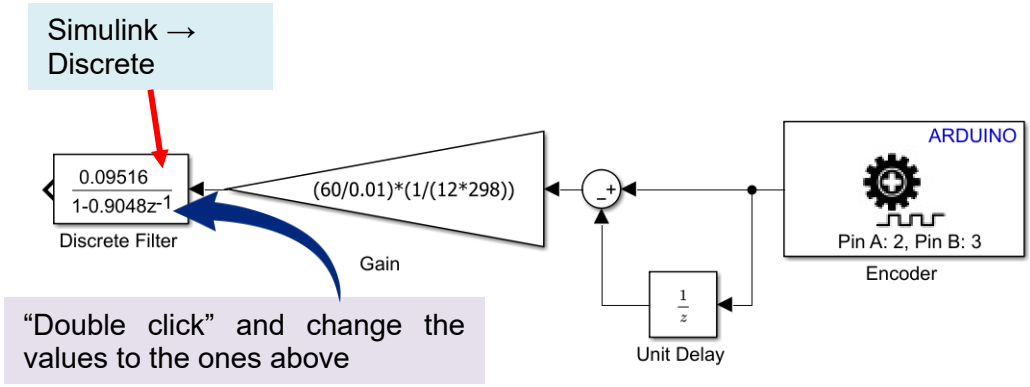


Figure 3: *Simulink Implementation of Encoder-Based Motor Speed Measurement and Low-Pass Filtering*

A Pulse Generator and Sum block are now introduced to define the reference speed and prepare the model for closed-loop control, see Figure 4. The Pulse Generator provides the desired motor speed, while the Sum block will later be used to calculate the difference between the reference and measured speeds.

The Pulse Generator is configured with a pulse type of sample based. An amplitude of 30 is given, corresponding to the requirement reference speed of 30RPM, see Table 2. A sample time of 0.01s is used, with a period of 1000 samples and pulse width of 500 samples, see Figure 4. This produces a 30RPM reference that remains active for 5 seconds before returning to zero. The 5-second pulse duration has been selected so that the motor response can be clearly observed against the 2-second rise-time requirement, see Table 2. This

makes it straightforward to identify visually whether the controlled motor reaches the required speed within the specified time.

A Pulse Width Modulation (PWM) block has now been added to provide the motor drive signal, see Figure 4, leaving only the PI controller to be configured.

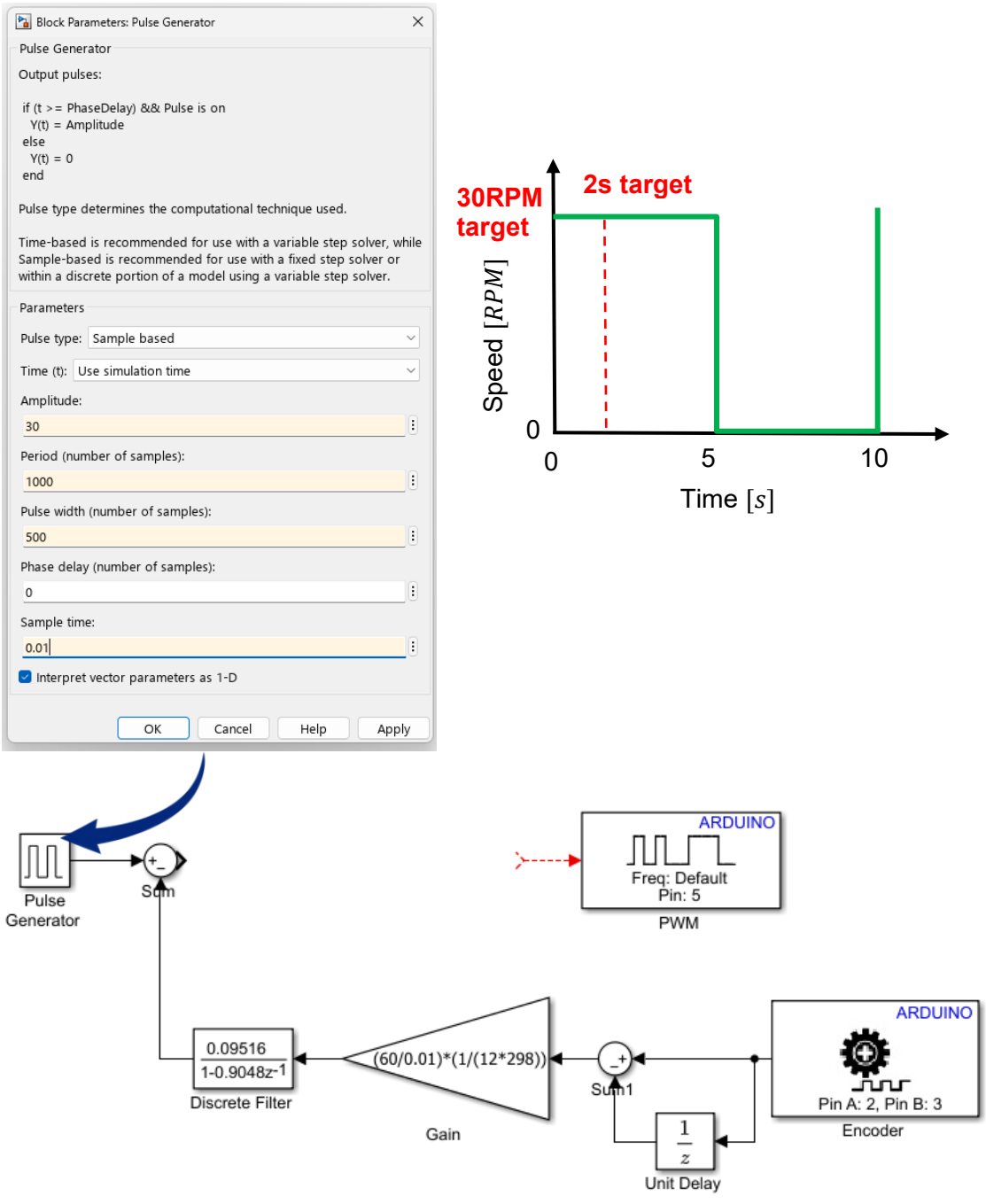


Figure 4: Simulink Set-Up Illustrating the 30RPM Reference and Generation of the Error Signal Using a Summing Junction

As shown in Figure 5 (a), the PID Controller block is configured for a PI Controller and Discrete-time operation with a sample time of 0.01s. The trapezoidal method is selected for the discrete-time integrator, see Figure 5 (a). The proportional and integral gains are initially set to 1. These values provide a starting point for controller tuning, where the gains can be adjusted to achieve the specified motor-speed performance requirements that are given in Table 2. As shown in Figure 5 (b), saturation factors are considered. The PI controller output is limited to the practical PWM operating range of 0 to 170 (this is covered in Section 3.0 of the exercise: DC Motor Speed Measurement Using an Encoder). The upper limit of 170 ensures that the 6V motor is not driven beyond the permitted PWM command when supplied from 9V, while the lower limit of 0 reflects the one-direction motor-control configuration used in this exercise. Clamping anti-windup is selected to prevent the integral term from continuing to accumulate if the controller output reaches the saturation limit of 170.

To evaluate the engineering requirements, the relevant signals must first be captured from the Simulink model using Scope blocks. For requirements R1, R2, R3 and R5 listed in Table 2, the key signals include the reference speed, unfiltered motor speed, and filtered motor speed.

To route these signals without introducing unnecessary connecting lines across the model, Goto blocks are used, as shown in Figure 6. Corresponding From blocks then retrieve each signal and feed them into a common Scope, allowing the signals to be viewed and compared together. Additional Scope input ports can be created by selecting a From block and dragging its output connection towards the Scope, as illustrated in Figure 6. The Goto and From blocks can be found within the Signal Routing section of the Simulink Library Browser.

In addition, a separate Scope is connected to the controller output to determine whether the controller reaches its maximum allowable output, see Figure 6. This measurement is used to evaluate requirement R4 from Table 2.

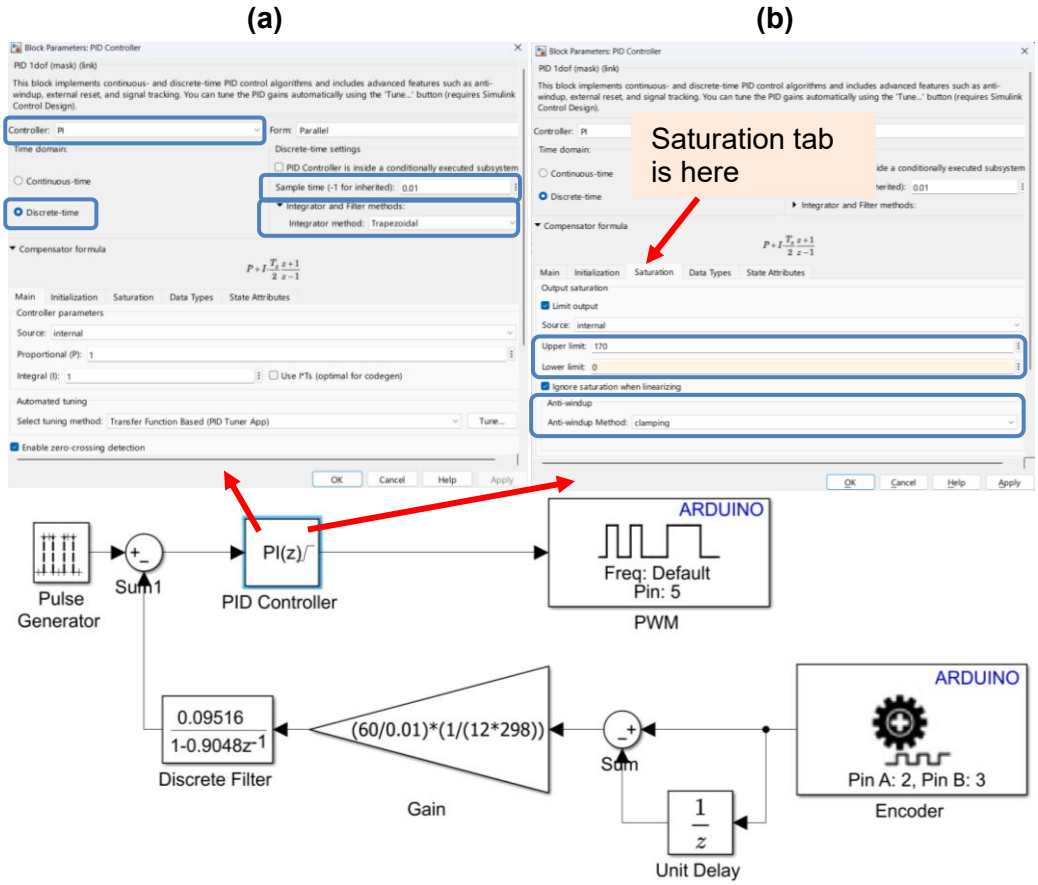


Figure 5: Simulink Model for DC Motor Speed Control using a PI Controller

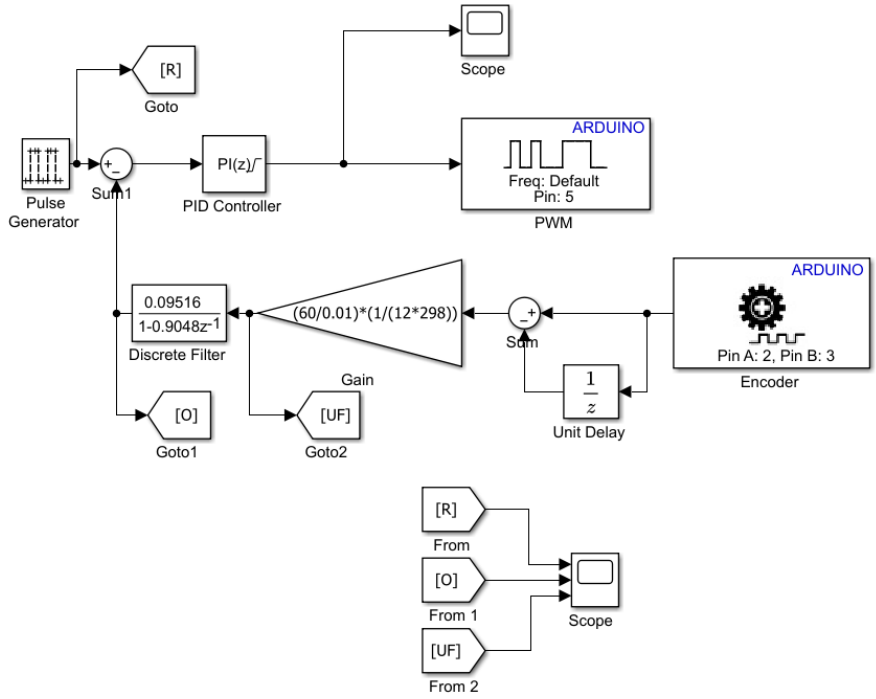


Figure 6: *Closed-Loop PI Motor Control System with Goto/From Signal Routing for Comparison of the Reference, Filtered and Unfiltered Speed Responses*

5.0 Verification and Validation

The complete Simulink model shown in Figure 6 should now be run using the normal procedure for deploying and executing the model on the Arduino hardware. Once the model is running, allow the motor to respond to the 30RPM reference input and use the Scope to observe the reference speed, unfiltered motor speed and filtered motor speed. The results you obtain should show a similar response to that presented in Figure 7.

The measured output response should then be compared with the engineering requirements defined earlier in the exercise. This provides the basis for verification, where the measured behaviour of the implemented control system is checked against the specified performance requirements.

Record the measured results in Table 3. **Appendix A** provides example results obtained from the same practical exercise for comparison. However, you should first determine and record the values from your own measured response.

Table 3: *Engineering Requirements Compared Against the Measured DC Motor Speed Response*

ID	Engineering Requirement	Measured Result?	Requirement Met?
R1	The DC motor shall track the specified reference speed of 30RPM.	_____RPM	Yes/No
R2	The closed-loop response shall achieve approximately zero steady-state error.	_____ RPM	Yes/No
R3	The rise time shall be less than 2s.	_____s	Yes/No
R4	The controller output shall remain within the permitted actuator range of 0–170, i.e., the permitted actuator range.	_____	Yes/No
R5	The measured speed signal shall be sufficiently filtered to enable stable closed-loop control.	_____	Yes/No

Following verification, consider the overall behaviour of the closed-loop system. This forms the validation stage, where the suitability of the implemented control system is considered more broadly.

Comment briefly on the following:

- Does the motor respond appropriately to the 30RPM reference input?
- Does the filtered speed signal provide a sufficiently smooth representation of the motor response?
- Does the closed-loop response appear stable?
- Are all of the engineering requirements satisfied?
- If not, what aspects of the controller may require further development?

If one or more requirements are not achieved, the results should be used to identify where further development is required. This may include adjustment of the PI controller gains, refinement of the low-pass filter design, or both.

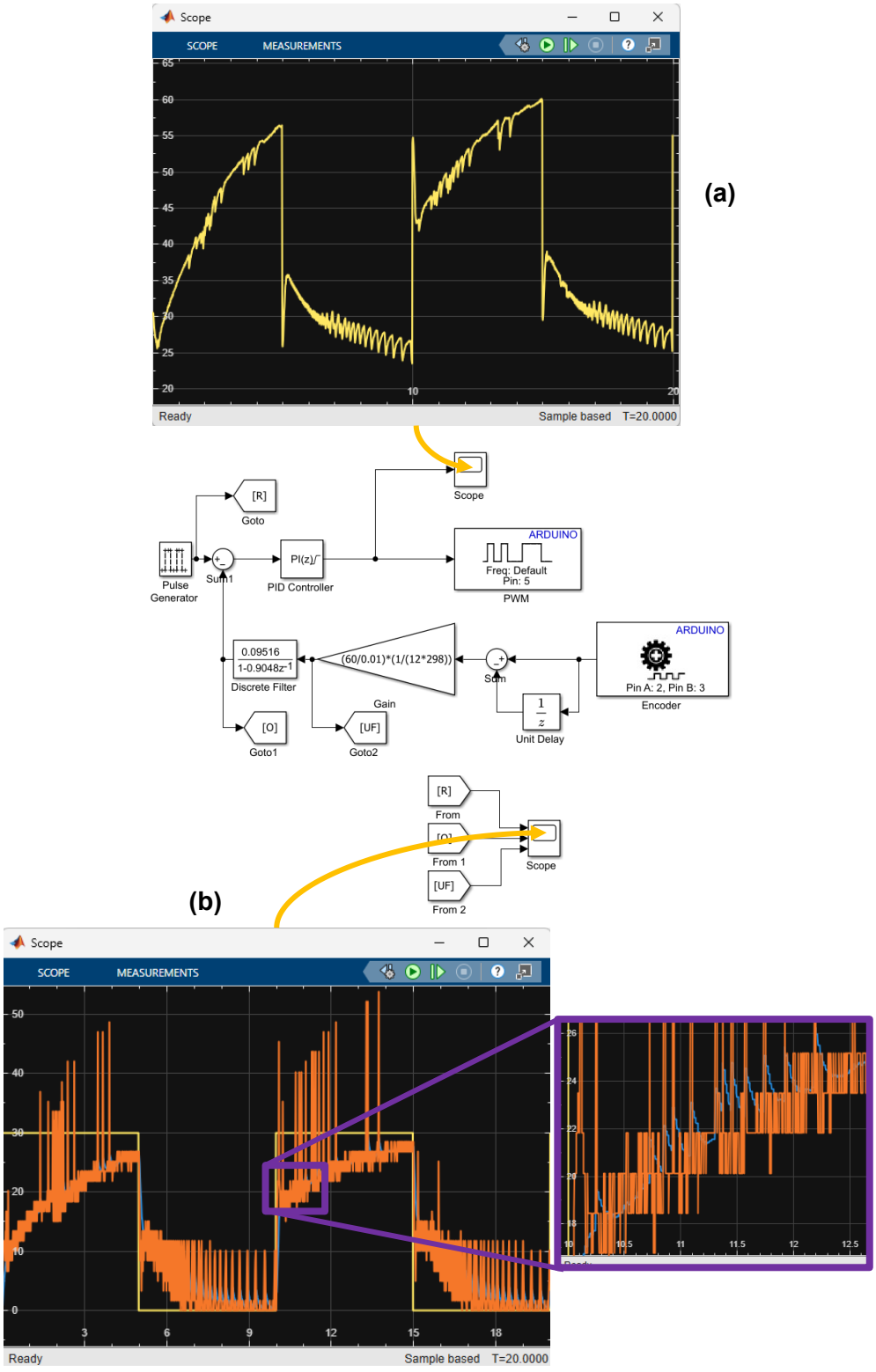


Figure 7: *PI Control Algorithm Design Outputs: (a) Control Output (b) Reference Unfiltered Output and Filtered Output*

6.0 Concluding Remarks

This practical exercise has introduced the implementation and evaluation of a closed-loop PI controller for DC motor speed control using Arduino, Simulink and a DC motor with encoder feedback. The controller was implemented in discrete time and incorporated output saturation, anti-windup and low-pass filtering of the measured motor speed.

The measured closed-loop response was then compared against the engineering requirements defined at the beginning of the exercise. This verification and validation process provides an important part of the engineering workflow, allowing the performance of the implemented system to be assessed and areas requiring further development to be identified. Where requirements were not achieved, potential improvements may include further tuning of the PI controller gains, refinement of the low-pass filter design, or both.

Before completing the exercise, return to the Learning Outcomes in Table 1 and reflect on whether each has been achieved. If you are confident that you can demonstrate a Learning Outcome, tick the corresponding *Achieved* box in Table 1.

7.0 Equipment Pack Away

After completing the practical exercise, the ACE-Lab equipment should be disconnected and returned to the appropriate storage trays ready for the next exercise.

Before dismantling the hardware, ensure that the Simulink model has stopped running. Disconnect the 9 V battery from the Maker Drive motor driver first, followed by the USB Type-B cable from the Arduino. The remaining jumper wires and hardware components can then be carefully disconnected.

Return all components to their designated locations within the ACE-Lab (Base + Sense) and ACE-Lab (Actuate) trays, as illustrated in Figure 8. Ensure that the jumper wires are grouped together and that the DC motor, Maker Drive motor driver, battery connector and other components are securely positioned within the appropriate compartments.

Do not leave the 9 V battery connected to the motor driver after completing the exercise.

Before packing away the ACE-Lab, check that all components used during the exercise have been returned to the appropriate tray.

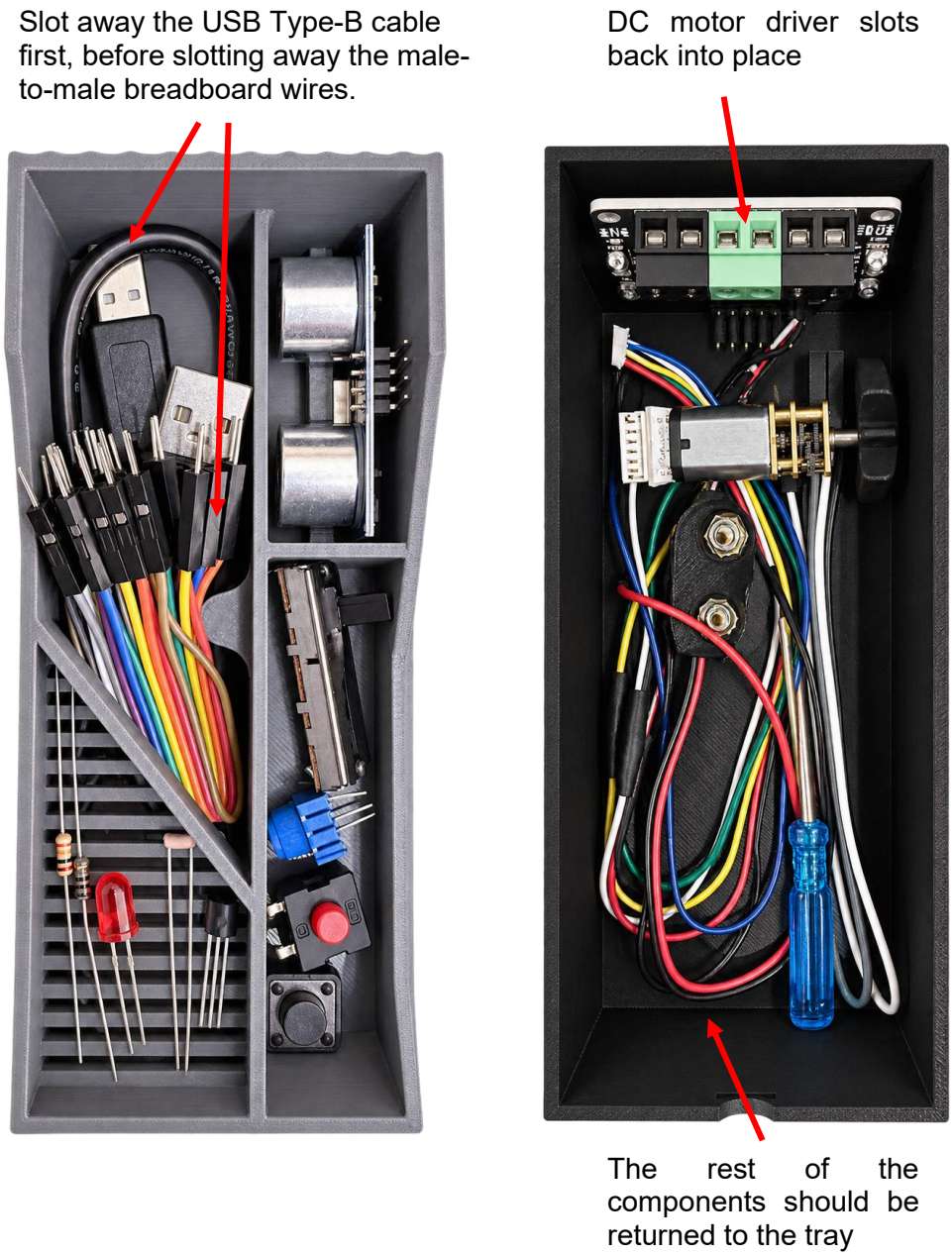


Figure 8: Tray *Arrangement of Components When Packing Away the ACE-Lab (Base + Sense, Actuate)*

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

> Pickering, J. E. (2026). DC Motor Speed PI Control. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/23043611.

Example results from the practical experiment are given in Table A-1.

Table A-1: Example Results Compared Against the Engineering Requirements for the DC Motor Speed Control System

ID	Engineering Requirement	Measured Result?	Requirement Met?
R1	The DC motor shall track the specified reference speed of 30RPM.	_~27_RPM	No
R2	The closed-loop response shall achieve approximately zero steady-state error.	_~3_RPM	No
R3	The rise time shall be less than 2s.	_~4.5__s	No
R4	The controller output shall remain within the permitted actuator range of 0–170, i.e., the permitted actuator range.	__~60___	Yes
R5	The measured speed signal shall be sufficiently filtered to enable stable closed-loop control.	__ Insufficient filtering; measured response remains noisy___	No